

智能仓储分拣系统

学生团队项目需求文档

团队规模	10人
分组	A组(传感器3人) B组(后端2人) C组(异常监测2人) D组(可视化3人)
技术栈	实物设备 + MQTT通信(基于Socket) + openGauss + 麒麟OS + Unity3D + YOLO
开发环境	笔记本电脑(服务器用虚拟机)

文档版本: v1.0 编写日期: 2025年6月15日 下次评审: 2025年6月30日

智能仓储分拣系统 — 学生团队项目需求文档

团队规模：10人

分组：A组(传感器3人) | B组(后端2人) | C组(异常监测2人) | D组(可视化3人)

技术栈：实物设备 + MQTT协议 + Mosquitto Broker + openGauss + 麒麟OS + Unity3D + YOLO

开发环境：笔记本电脑（服务器用虚拟机）

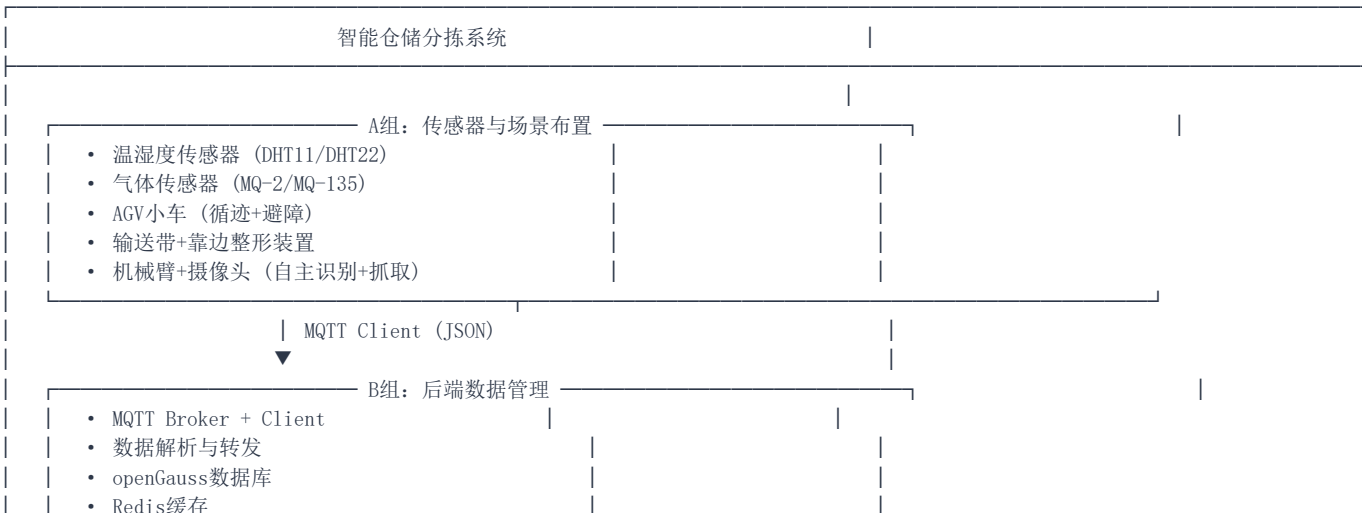
里程碑：6月30日前完成需求确认与环境搭建，暑假正式开始开发

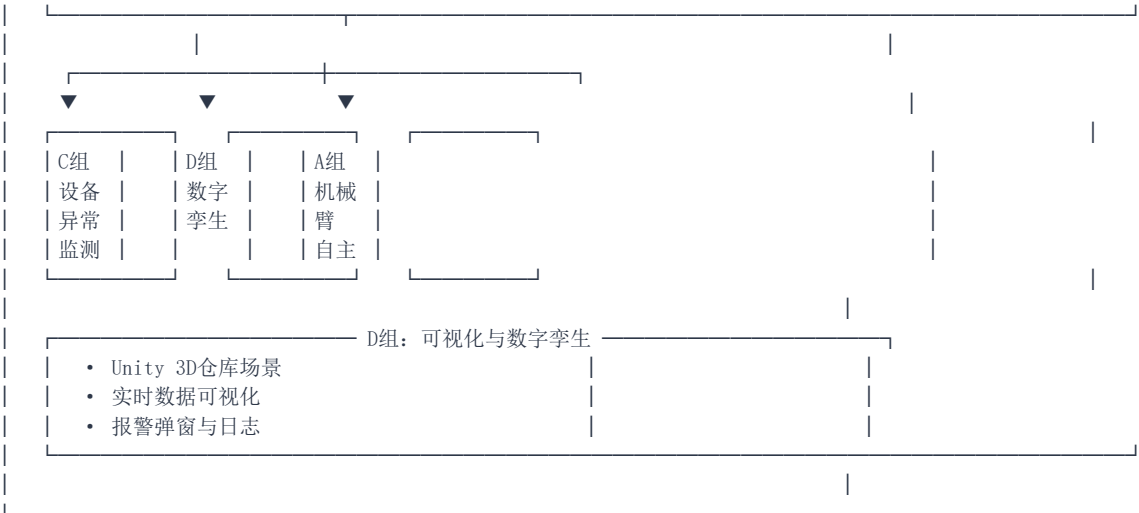
目录

- 项目总体架构
- A组：传感器与场景布置组（3人）
- B组：信号传输与后端数据管理组（2人）
- C组：异常监测与报警组（2人）
- D组：可视化与数字孪生组（3人）
- 组间协作规范
- 数据格式约定
- 并行开发计划
- 开发里程碑
- 风险与应对

一、项目总体架构

1.1 系统架构图





1.2 硬件清单（学生版）

设备	数量	负责组	预算参考	说明
笔记本电脑	10台	全员	自备	开发+部署+虚拟机
温湿度传感器DHT22	4个	A组	¥40	仓库四角部署
气体传感器MQ-2	2个	A组	¥30	检测烟雾/有害气体
Arduino/ESP32	2套	A组	¥100	传感器采集+MQTT发送
AGV小车套件	1-2辆	A组	¥300-600	循迹小车，带红外避障
机械臂（4DOF）	1台	A组	¥400-800	带摄像头自主识别抓取
USB摄像头	1个	A组	¥50	装在机械臂上用于识别
输送带套件	1套	A组	¥200	或自制简易输送带
路由器	1个	A组	¥100	局域网搭建
合计	-	-	¥1500-2000	可根据预算调整

1.3 软件环境

组件	版本/说明	部署位置
麒麟OS虚拟机	V10	B组笔记本 (VMware/VirtualBox)
openGauss	3. 0+	麒麟OS虚拟机
Redis	6. 0+	B组笔记本
Python	3. 8+	全员
Unity	2021 LTS	D组笔记本
OpenCV	4. 5+	A组/B组/C组
YOLOv5/v8	预训练模型	A组笔记本（机械臂识别）

二、A组：传感器与场景布置组（3人）



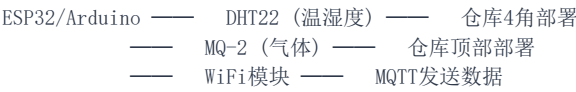
2.1 组内分工

角色	人数	职责
传感器开发	1人	温湿度/气体传感器采集程序
AGV控制	1人	小车循迹、避障、通信
机械臂开发	1人	机械臂视觉识别、抓取控制、通信

2.2 核心任务

任务1：环境传感器系统

硬件连接：



软件功能：

- 每5秒采集一次温湿度、气体浓度
- 通过WiFi以MQTT Client方式发送JSON数据到B组
- 本地OLED显示当前数值
- 支持断线重连

输出数据格式：

```
{
  "msg_type": "sensor_data",
  "device_id": "ENV-01",
  "device_type": "env_sensor",
  "timestamp": "2025-06-15T10:30:00+08:00",
  "payload": {
    "temperature": 26.5,
    "humidity": 65.0,
    "gas_ppm": 150
  },
  "seq": 1001
}
```

任务2：AGV小车系统

功能需求：

- 红外循迹（黑线跟踪）
- 超声波避障（前方障碍物检测）
- 通过 MQTT 订阅 control/agv/{deviceId} 接收调度指令
- 自动返回充电（模拟，到达指定位置即可）

通信协议：

```
// AGV发送状态（每500ms）
{
  "msg_type": "agv_status",
```

```
"device_id": "AGV-01",
"device_type": "agv",
"timestamp": "2025-06-15T10:30:00+08:00",
"payload": {
  "position": "SORT-AREA-1",
  "position_x": 1.2,
  "position_y": 0.8,
  "battery": 85,
  "status": "running",
  "task_id": "TASK-001"
},
"seq": 2001
}

// AGV接收指令（来自B组）
{
  "msg_type": "control_command",
  "command_id": "CMD-002",
  "source": "B组调度服务",
  "target_device": "AGV-01",
  "device_type": "agv",
  "command": "agv_move_to",
  "params": {
    "target": "OUT-AREA-1"
  },
  "timestamp": "2025-06-15T10:30:00+08:00",
  "require_confirm": true,
  "triggered_by_alert": null
}
```

任务3：机械臂自主识别与抓取系统

系统架构：

机械臂 + 摄像头（装在机械臂末端或上方）

- |
- |—— 摄像头采集图像
- |—— YOLO识别货物位置和类别
- |—— 计算抓取坐标（相对于机械臂）
- |—— 执行抓取动作
- |—— 放置到对应分拣区
- |—— 发送动作位置给B组后端

功能需求：



自主识别：使用YOLOv5s预训练模型，识别3类货物（书book、瓶子bottle、盒子box）



坐标转换：将图像坐标转换为机械臂抓取坐标



抓取控制：根据识别结果执行抓取动作



分类放置：根据货物类别放置到对应分拣区（A区/B区/未知区）



状态上报：将动作位置、抓取结果发送给B组后端

简化策略（降低难度）：



使用固定点位抓取（3-5个预设位置）



摄像头固定在机械臂末端或上方固定位置



识别到货物后，移动到最近的预设抓取点



分拣区固定为3个位置（对应3种货物类型）

通信协议（发送给B组）：

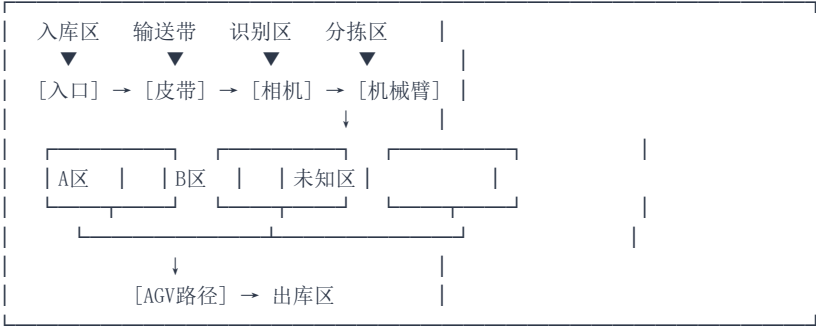
```
// 机械臂动作上报（每次动作完成后发送）
{
  "msg_type": "robot_action",
  "device_id": "ROBOT-01",
  "device_type": "robot",
  "timestamp": "2025-06-15T10:30:00+08:00",
  "payload": {
    "action": "grasp_complete",
    "cargo_type": "box",
    "confidence": 0.92,
    "source_position": "CONVEYOR-1",
    "target_position": "SORT-A",
    "grasp_success": true,
    "joint_angles": [0, 45, 90, 0],
    "tcp_position": [120, 80, 50],
    "error_code": 0
  },
  "seq": 3001
}
```

技术实现:

- 使用Python + OpenCV + YOLOv5（轻量版）
- 在A组笔记本上运行识别程序
- 通过串口/USB控制机械臂
- 通过WiFi发送数据给B组

任务4：场景物理搭建

场景布局（建议尺寸：2m × 1.5m）：



与D组协作:

- 提供场景尺寸给D组用于Unity建模
- 确定坐标系原点和方向
- 标记关键点位置（入库点、分拣点、出库点）

2.3 交付物

交付物	截止时间	验收标准
传感器采集程序	7月5日	能稳定采集并MQTT发送数据
AGV控制程序	7月10日	能循迹、避障、接收指令
机械臂识别+控制程序	7月20日	能自主识别、抓取、上报位置
物理场景搭建	7月25日	完整可运行

交付物	截止时间	验收标准
技术文档	7月25日	接线图、通信协议说明

2.4 开发提示

- 并行开发策略：
- 前期没有B组后端时，使用Python写一个简单的MQTT Broker模拟后端
 - 传感器可以用手遮挡/加热/吹气模拟数值变化
 - AGV可以先在桌面上测试循迹
 - 机械臂可以先手动控制测试动作

三、B组：信号传输与后端数据管理组（2人）



3.1 组内分工

角色	人数	职责
MQTT通信服务端	1人	接收所有传感器数据、数据解析转发
数据库管理	1人	openGauss部署、数据存储、API开发

3.2 核心任务

任务1：MQTT通信服务（Mosquitto Broker + paho-mqtt）

- 功能需求：
- Mosquitto Broker部署（端口1883/9001）
 - 订阅 sensor/# 主题，接收A组传感器数据
 - 数据解析、校验、格式转换
 - 转发到Redis和openGauss
 - 支持多客户端并发（≥10个连接）
- 技术选型：
- paho-mqtt客户端订阅Topic + FastAPI提供HTTP/WebSocket接口
 - 数据流转：Mosquitto → paho-mqtt订阅 → FastAPI → openGauss/Redis → WebSocket推送
- 关键代码框架：
- ```
mqtt_listener.py — MQTT订阅 + FastAPI服务框架
import paho.mqtt.client as mqtt
import json
from fastapi import FastAPI
from fastapi import WebSocket

MQTT回调：收到 sensor/# 数据
def on_message(client, userdata, msg):
 data = json.loads(msg.payload)
 # 存入Redis（实时缓存）
```

```
redis_client.set(f"device:{data['device_id']}", json.dumps(data))
存入openGauss（历史存储）
db.insert_device_data(data)
WebSocket推送给D组可视化
ws_manager.broadcast("sensor_data", data)

初始化MQTT客户端
client = mqtt.Client()
client.on_message = on_message
client.connect("localhost", 1883, 60)
client.subscribe("sensor/#", qos=1)
client.loop_start()

FastAPI服务（供D组查询 + WebSocket）
app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(ws: WebSocket):
 await ws_manager.connect(ws)
 while True:
 data = await ws.receive_json()
 # 收到D组控制指令 → 发布到 control/#
 # 提取设备类型和设备ID，构造标准Topic
 device_type = data.get("device_type", "unknown")
 target_device = data.get("target_device", "unknown")
 topic = f"control/{device_type}/{target_device}"
 client.publish(topic, json.dumps(data), qos=1)
```

## 任务2：openGauss数据库

部署环境：

- 麒麟OS虚拟机（B组笔记本安装VMware/VirtualBox）
- 分配资源：2核CPU、4GB内存、20GB磁盘

数据库设计：

— 设备注册表

```
CREATE TABLE devices (
 device_id VARCHAR(32) PRIMARY KEY,
 device_type VARCHAR(32), -- sensor/agv/robot/camera
 device_name VARCHAR(64),
 location VARCHAR(64),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

— 传感器数据表

```
CREATE TABLE sensor_data (
 id SERIAL PRIMARY KEY,
 device_id VARCHAR(32),
 temperature FLOAT,
 humidity FLOAT,
 gas_ppm FLOAT,
 timestamp TIMESTAMP,
 FOREIGN KEY (device_id) REFERENCES devices(device_id)
);
```

— AGV数据表

```
CREATE TABLE agv_data (
 id SERIAL PRIMARY KEY,
 device_id VARCHAR(32),
 position VARCHAR(32),
 position_x FLOAT,
```



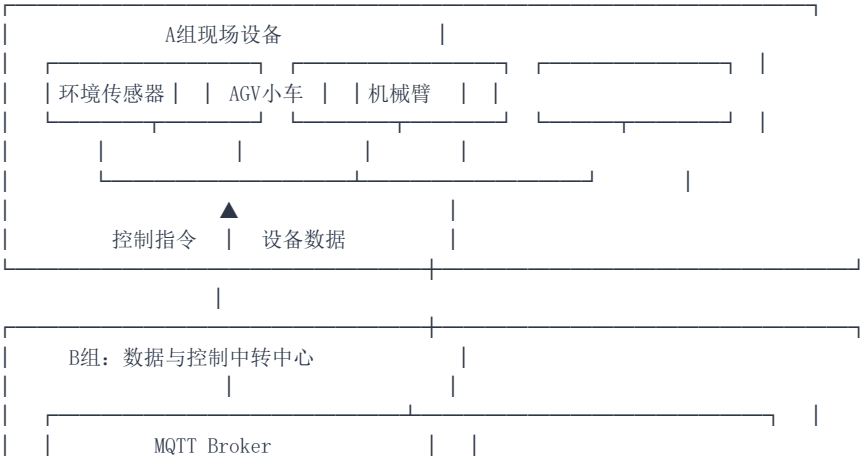
```
position_y FLOAT,
battery INT,
status VARCHAR(32),
task_id VARCHAR(32),
seq INT,
raw_payload JSON,
timestamp TIMESTAMP
);

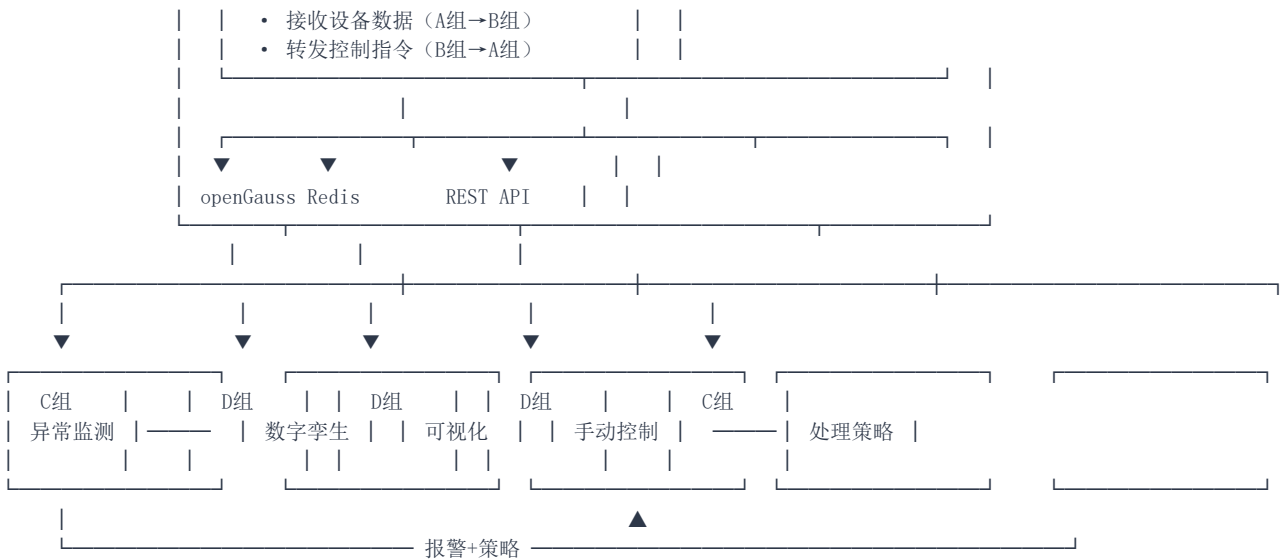
-- 货物识别记录表
CREATE TABLE cargo_recognition (
 id SERIAL PRIMARY KEY,
 cargo_id VARCHAR(32),
 cargo_type VARCHAR(32),
 confidence FLOAT,
 position_x FLOAT,
 position_y FLOAT,
 timestamp TIMESTAMP
);

-- 报警记录表
CREATE TABLE alerts (
 id SERIAL PRIMARY KEY,
 alert_type VARCHAR(32),
 alert_level INT, -- 1-5
 device_id VARCHAR(32),
 message TEXT,
 status VARCHAR(32), -- active/resolved
 created_at TIMESTAMP,
 resolved_at TIMESTAMP
);
```

任务3：数据接收与设备管理服务

- 功能说明：
- B组不再负责YOLO识别（由A组机械臂自主完成）
  - B组专注于数据接收、存储、转发、控制指令中转
  - 接收A组机械臂上报的动作位置数据
  - 接收A组传感器数据、AGV数据
  - 接收C组处理策略、D组控制指令，转发给A组执行
  - 提供统一的数据查询API
- 数据流（双向）：





<b>机械臂数据接收处理</b>:

```
接收机械臂上报数据示例
def handle_robot_data(data):
 """
 接收A组机械臂上报的动作数据
 """
 robot_data = {
 'device_id': data['device_id'],
 'action': data['payload']['action'],
 'cargo_type': data['payload']['cargo_type'],
 'confidence': data['payload'].get('confidence'),
 'source_position': data['payload']['source_position'],
 'target_position': data['payload']['target_position'],
 'grasp_success': data['payload']['grasp_success'],
 'joint_angles': data['payload']['joint_angles'],
 'tcp_position': data['payload'].get('tcp_position'),
 'error_code': data['payload'].get('error_code'),
 'seq': data.get('seq'),
 'raw_payload': data,
 'timestamp': data['timestamp']
 }

 # 存入数据库
 db.insert('robot_actions', robot_data)

 # 发布给C组（异常监测）和D组（可视化）
 redis.publish('robot_data', json.dumps(robot_data))
```

<b>新增数据库表</b>:

```
-- 机械臂动作记录表
CREATE TABLE robot_actions (
 id SERIAL PRIMARY KEY,
 device_id VARCHAR(32),
 action VARCHAR(32),
 cargo_type VARCHAR(32),
 confidence FLOAT,
 source_position VARCHAR(32),
 target_position VARCHAR(32),
 grasp_success BOOLEAN,
 joint_angles JSON,
 tcp_position JSON,
 error_code INT,
 seq INT,
```

```
raw_payload JSON,
timestamp TIMESTAMP
);

-- 分拣统计表（实时更新）
CREATE TABLE sort_statistics (
 id SERIAL PRIMARY KEY,
 cargo_type VARCHAR(32),
 total_count INT DEFAULT 0,
 success_count INT DEFAULT 0,
 fail_count INT DEFAULT 0,
 last_updated TIMESTAMP
);
```

任务4: REST API开发

API列表:

| 接口                     | 方法       | 说明              |
|------------------------|----------|-----------------|
| /api/devices           | GET      | 获取设备列表          |
| /api/sensor/latest     | GET      | 获取最新传感器数据       |
| /api/sensor/history    | GET      | 获取传感器历史数据       |
| /api/agv/status        | GET      | 获取AGV状态         |
| /api/agv/command       | POST     | 发送AGV控制指令       |
| /api/robot/command     | POST     | 发送机械臂控制指令       |
| /api/env/control       | POST     | 控制环境设备（风扇/蜂鸣器等） |
| /api/cargo/recognition | GET      | 获取货物识别记录        |
| /api/alerts            | GET      | 获取报警列表（含处理策略）   |
| /api/alerts/ack        | POST     | 确认报警            |
| /api/alerts/execute    | POST     | 执行处理策略          |
| /api/control/forward   | POST     | 转发控制指令到设备       |
| /api/mode              | GET/POST | 获取/设置系统运行模式     |
| /api/statistics/sort   | GET      | 获取分拣统计          |
| /api/statistics/alerts | GET      | 获取异常统计          |
| /api/devices/history   | GET      | 获取设备状态历史        |
| /api/alerts            | GET      | 获取报警列表          |
| /api/alerts/ack        | POST     | 确认报警            |

3.3 交付物

| 交付物         | 截止时间  | 验收标准             |
|-------------|-------|------------------|
| MQTT Broker | 7月5日  | 能接收并存储数据         |
| openGauss部署 | 7月10日 | 数据库正常运行，表结构完成    |
| 设备数据管理服务    | 7月15日 | 能处理机械臂/AGV/传感器数据 |
| 控制指令转发服务    | 7月18日 | 能接收并转发控制指令到A组    |
| REST API    | 7月22日 | 所有接口可用，有文档       |

| 交付物  | 截止时间  | 验收标准       |
|------|-------|------------|
| 技术文档 | 7月25日 | API文档、部署手册 |

3.4 开发提示

- 并行开发支持:
- 提供模拟数据生成器（Python脚本），让C/D组前期有数据可用
  - 使用SQLite本地数据库过渡，等麒麟OS环境就绪后迁移
  - API先用Flask快速搭建，后期可优化

四、C组：异常监测与报警组（2人）



4.1 组内分工

| 角色     | 人数 | 职责                   |
|--------|----|----------------------|
| 设备异常监测 | 1人 | 监测机械臂、AGV、传感器等设备运行状态 |
| 报警与联动  | 1人 | 报警逻辑、环境联动控制、通知推送     |

4.2 核心任务

任务1：设备运行状态监测

- 监测对象:
- 机械臂：关节角度异常、抓取失败率、运行状态
  - AGV：电量、位置偏差、避障触发、离线状态
  - 环境传感器：温湿度超限、气体浓度、离线状态
- 机械臂异常检测规则:

| 异常类型   | 触发条件       | 等级 | 说明      |
|--------|------------|----|---------|
| 关节角度异常 | 关节角度超出预设范围 | L3 | 机械臂姿态异常 |
| 抓取失败率高 | 连续3次抓取失败   | L3 | 识别或控制问题 |
| 机械臂离线  | 1分钟无数据上报   | L4 | 通信中断    |
| 放置位置错误 | 目标位置与预期不符  | L3 | 分拣错误    |

机械臂坐标异常检测代码框架:

```
robot_anomaly_detector.py
class RobotAnomalyDetector:
 def __init__(self):
 self.joint_limits = {
 'joint1': (-90, 90),
 'joint2': (0, 180),
 'joint3': (0, 180),
 'joint4': (-90, 90)
 }
```

```
self.grasp_fail_count = 0
self.last_data_time = None

def check_joint_angles(self, joint_angles):
 """检查关节角度是否在正常范围内"""
 for i, angle in enumerate(joint_angles):
 joint_name = f'joint{i+1}'
 min_val, max_val = self.joint_limits[joint_name]
 if angle < min_val or angle > max_val:
 return {
 'alert_level': 3,
 'alert_type': 'robot_joint_abnormal',
 'message': f'{joint_name}角度异常: {angle}° ',
 'device_id': 'ROBOT-01'
 }
 return None

def check_grasp_success(self, grasp_success):
 """检查抓取成功率"""
 if not grasp_success:
 self.grasp_fail_count += 1
 if self.grasp_fail_count >= 3:
 return {
 'alert_level': 3,
 'alert_type': 'robot_grasp_fail',
 'message': f'连续{self.grasp_fail_count}次抓取失败',
 'device_id': 'ROBOT-01'
 }
 else:
 self.grasp_fail_count = 0
 return None

def check_offline(self, last_timestamp):
 """检查机械臂是否离线"""
 import time
 from datetime import datetime, timezone, timedelta

 last_time = datetime.fromisoformat(last_timestamp)
 current_time = datetime.now(timezone(timedelta(hours=8)))
 diff = (current_time - last_time).total_seconds()

 if diff > 60: # 1分钟无数据
 return {
 'alert_level': 4,
 'alert_type': 'robot_offline',
 'message': '机械臂离线超过1分钟',
 'device_id': 'ROBOT-01'
 }
 return None
```

<b>AGV异常检测规则</b>:

| 异常类型  | 触发条件             | 等级 |
|-------|------------------|----|
| 电量低   | 电量 < 20%         | L2 |
| 位置偏差  | 实际位置与目标位置偏差>10cm | L3 |
| 避障触发  | 超声波检测到障碍物        | L3 |
| AGV离线 | 1分钟无数据           | L3 |

<b>环境异常检测规则</b>:

| 异常类型  | 触发条件          | 等级 |
|-------|---------------|----|
| 高温    | 温度 > 30℃      | L2 |
| 高湿    | 湿度 > 75%      | L2 |
| 气体超标  | 气体浓度 > 300ppm | L3 |
| 传感器离线 | 5分钟无数据        | L3 |

任务2：异常报警与处理策略生成

报警分级与处理策略：

| 等级 | 名称 | 典型场景           | 自动处理策略      | 需人工确认 |
|----|----|----------------|-------------|-------|
| L1 | 提示 | 湿度略高、AGV电量偏低   | 记录日志        | 否     |
| L2 | 注意 | 温度偏高、AGV电量<20% | 黄灯提示，建议充电   | 否     |
| L3 | 警告 | 机械臂抓取失败、传感器离线  | 红灯+蜂鸣器，暂停设备 | 是     |
| L4 | 严重 | 机械臂离线、气体超标     | 红灯+蜂鸣器+启动排风 | 是     |
| L5 | 紧急 | 设备碰撞、急停        | 全停+声光报警     | 必须    |

处理策略生成逻辑：

```
strategy_generator.py
class StrategyGenerator:
 def generate_strategy(self, alert):
 """根据报警类型生成处理策略"""
 strategies = {
 'robot_joint_abnormal': {
 'auto_action': 'robot_pause',
 'manual_options': [
 {'label': '复位机械臂', 'command': 'robot_reset'},
 {'label': '忽略继续', 'command': 'robot_resume'},
 {'label': '紧急停止', 'command': 'robot_stop'}
],
 'description': '机械臂关节角度异常，建议复位或检查'
 },
 'robot_grasp_fail': {
 'auto_action': 'robot_pause',
 'manual_options': [
 {'label': '重新抓取', 'command': 'robot_retry'},
 {'label': '跳过该货物', 'command': 'robot_skip'},
 {'label': '人工接管', 'command': 'robot_manual'}
],
 'description': '连续抓取失败，建议检查货物位置或人工接管'
 },
 'high_temperature': {
 'auto_action': 'fan_on',
 'manual_options': [
 {'label': '启动风扇', 'command': 'fan_on'},
 {'label': '启动排风', 'command': 'exhaust_on'},
 {'label': '暂停作业', 'command': 'system_pause'}
],
 'description': '温度过高，已自动启动风扇，可选择其他措施'
 },
 'agv_low_battery': {

```

```
 'auto_action': 'agv_charge',
 'manual_options': [
 {'label': '立即充电', 'command': 'agv_charge_now'},
 {'label': '完成任务后充电', 'command': 'agv_charge_later'},
 {'label': '更换AGV', 'command': 'agv_switch'}
],
 'description': 'AGV电量不足, 建议充电或更换'
 },
 'gas_alert': {
 'auto_action': 'exhaust_on',
 'manual_options': [
 {'label': '加大排风', 'command': 'exhaust_high'},
 {'label': '暂停作业', 'command': 'system_pause'},
 {'label': '人员撤离', 'command': 'evacuate'}
],
 'description': '气体浓度超标, 已启动排风, 建议检查气体源'
 }
}

alert_type = alert['alert_type']
if alert_type in strategies:
 return strategies[alert_type]
else:
 return {
 'auto_action': 'none',
 'manual_options': [{'label': '确认', 'command': 'ack'}],
 'description': '未知异常, 请人工检查'
 }
```

### 报警与策略推送流程:

# 异常处理完整流程

```
def handle_anomaly(device_data):
 # 1. 检测异常
 alert = detector.check(device_data)
 if not alert:
 return

 # 2. 生成处理策略
 strategy = strategy_generator.generate_strategy(alert)
 alert['strategy'] = strategy
 alert['mode'] = 'auto' # 默认自动模式

 # 3. 执行自动策略 (L3及以上暂停, 等待D组确认)
 if alert['alert_level'] >= 3:
 execute_auto_action(strategy['auto_action'])
 alert['status'] = 'waiting_confirm' # 等待D组确认
 else:
 alert['status'] = 'auto_resolved'

 # 4. 保存到数据库
 db.save_alert(alert)

 # 5. 推送到B组, 由B组转发给D组
 redis.publish('alerts', json.dumps(alert))

 return alert
```

## 任务3: 报警通知系统

### 报警通知方式:

- 数据库存储: 所有报警记录到alerts表

- <b>Redis发布</b>: 实时推送到D组可视化
- <b>声光报警</b>: 控制蜂鸣器、LED灯（A组硬件）
- <b>日志记录</b>: 本地日志文件便于追溯

<b>报警状态流转</b>:

触发(triggered) → 活跃(active) → 确认(acknowledged) → 解决(resolved)  
|  
└─ 自动恢复（如温度恢复正常）→ 已恢复(resolved)

<b>通知代码框架</b>:

```
alert_notifier.py
class AlertNotifier:
 def __init__(self):
 self.redis_client = redis.Redis()
 self.db = Database()

 def notify(self, alert):
 """发送报警通知"""
 # 1. 存入数据库
 alert_id = self.db.insert_alert(alert)

 # 2. 发布到Redis（D组订阅）
 self.redis_client.publish('alerts', json.dumps(alert))

 # 3. 声光报警（根据等级）
 if alert['alert_level'] >= 3:
 self.trigger_buzzer(alert['alert_level'])

 # 4. 记录日志
 logging.warning(f"报警: {alert['message']}")

 def trigger_buzzer(self, level):
 """触发蜂鸣器"""
 if level >= 4:
 # 高频急促
 send_command('BUZZER-01', 'high')
 elif level == 3:
 # 中频
 send_command('BUZZER-01', 'medium')
```

## 任务4：报警系统

<b>报警方式</b>:

- <b>写入数据库</b>: 所有报警记录到alerts表
- <b>Redis发布</b>: 实时推送到D组
- <b>日志记录</b>: 本地日志文件
- <b>声光报警</b>: 控制蜂鸣器（A组硬件）

<b>报警状态流转</b>:

触发 → 活跃(active) → 确认(acknowledged) → 解决(resolved)  
↓  
自动恢复（如温度恢复正常）

## 4.3 交付物



| 交付物      | 截止时间  | 验收标准             |
|----------|-------|------------------|
| 设备异常检测程序 | 7月10日 | 能检测机械臂/AGV/传感器异常 |
| 报警联动程序   | 7月15日 | 能控制风扇/蜂鸣器/LED    |
| 报警通知系统   | 7月20日 | 报警记录、推送、可视化正常    |
| 技术文档     | 7月25日 | 异常规则说明、联动逻辑      |

4.4 开发提示

- 并行开发策略：
- 前期使用B组提供的模拟数据测试检测逻辑
  - 阈值可以先设宽松，后期根据实际数据调整
  - 联动控制先用print代替真实硬件控制，等A组就绪后对接

五、D组：可视化与数字孪生组（3人）



5.1 组内分工

| 角色      | 人数 | 职责                       |
|---------|----|--------------------------|
| Unity开发 | 2人 | 3D场景搭建、自动巡检动画、数据绑定、摄像头集成 |
| UI/前端   | 1人 | 数据面板、报警处理、手动控制界面、数据库可视化  |

5.2 核心任务

任务1：Unity 3D场景搭建

场景元素（与A组场景1:1对应）：

```
Warehouse Scene
├── 仓库建筑 (Warehouse Building)
│ ├── 地面 (Floor)
│ ├── 墙壁 (Walls)
│ └── 货架 (Shelves)
├── 设备层 (Equipment)
│ ├── 输送带 (Conveyor Belt) - 带动画
│ ├── 靠边整形装置 (Aligner)
│ ├── 机械臂 (Robot Arm) - 带动画
│ ├── AGV小车 (AGV) - 带动画+路径移动
│ └── 摄像头 (Camera)
├── 区域标记 (Zones)
│ ├── 入库区 (Inbound Zone)
│ ├── 识别区 (Recognition Zone)
│ ├── 分拣区A (Sort Zone A)
│ ├── 分拣区B (Sort Zone B)
│ ├── 未知货物区 (Unknown Zone)
│ └── 出库区 (Outbound Zone)
└── 传感器标记 (Sensors)
 ├── 温湿度传感器 (4个位置)
 └── 气体传感器 (2个位置)
```

简化策略（降低难度）：

- 使用Unity Asset Store免费资源
- 机械臂使用简化模型（6自由度→3自由度）
- AGV使用立方体+轮子表示
- 货物使用立方体/球体/圆柱体表示

任务2：数据可视化面板（新增功能）

主面板布局（扩展版）：

|                                                               |             |         |          |          |  |        |  |        |  |           |  |
|---------------------------------------------------------------|-------------|---------|----------|----------|--|--------|--|--------|--|-----------|--|
| 标题栏：智能仓储分拣系统 v1.0                                             |             |         |          | [自动模式 ▼] |  | [手动模式] |  |        |  |           |  |
| Unity 3D<br>场景视图<br>(占50%屏幕)<br><br>[自动巡检中...]<br><br>[摄像头画面] | 实时监控面板      |         | 数据库可视化   |          |  |        |  |        |  |           |  |
|                                                               | 环境参数        |         | • 分拣统计   |          |  |        |  |        |  |           |  |
|                                                               | • 温度：26℃    |         | • 异常记录   |          |  |        |  |        |  |           |  |
|                                                               | • 湿度：65%    |         | • 设备状态历史 |          |  |        |  |        |  |           |  |
|                                                               | • 气体：正常     |         | 异常监测可视化  |          |  |        |  |        |  |           |  |
|                                                               | 设备状态        |         | • 异常趋势图  |          |  |        |  |        |  |           |  |
|                                                               | • AGV-01：运行 |         | • 设备健康度  |          |  |        |  |        |  |           |  |
|                                                               | • 机械臂：待机    |         | • 报警统计   |          |  |        |  |        |  |           |  |
|                                                               |             |         | 手动控制区    |          |  |        |  |        |  |           |  |
|                                                               | 分拣统计        |         |          |          |  |        |  |        |  |           |  |
| • 今日：128件                                                     |             | [机械臂控制] |          |          |  |        |  |        |  |           |  |
| • 成功率：96%                                                     |             | [AGV调度] |          |          |  |        |  |        |  |           |  |
|                                                               |             | [环境设备]  |          |          |  |        |  |        |  |           |  |
| 底部：报警处理栏                                                      |             |         |          | [查看详情]   |  | [确认]   |  | [执行策略] |  | [切换自动/手动] |  |

新增功能模块：

- 运行模式切换
- 自动模式：C组算法自动处理异常，D组只监控
- 手动模式：D组可手动控制所有设备，处理异常
- 数据库可视化
- 实时显示分拣统计数据
- 异常记录查询和筛选
- 设备状态历史曲线
- 数据变化实时刷新
- 异常监测可视化
- 异常趋势图（按小时/天统计）
- 设备健康度评分
- 报警类型分布饼图
- 处理策略执行记录
- 手动控制区
- 机械臂控制：复位、暂停、继续、紧急停止

- AGV调度：指定位置、充电、切换
- 环境设备：风扇、排风机、蜂鸣器开关

### 任务3：实时数据绑定

数据获取方式：

- 从B组Redis订阅数据更新
- 或轮询B组REST API（每秒1次）

Unity数据绑定代码框架：

```
// DataManager.cs
using UnityEngine;
using System.Net.Http;
using System.Threading.Tasks;

public class DataManager : MonoBehaviour
{
 public string apiUrl = "http://192.168.1.100:8080/api";

 // 设备对象引用
 public GameObject agvObject;
 public GameObject robotArmObject;
 public TextMesh temperatureText;

 async void Update()
 {
 // 每秒获取一次数据
 if (Time.frameCount % 60 == 0)
 {
 await FetchData();
 }
 }

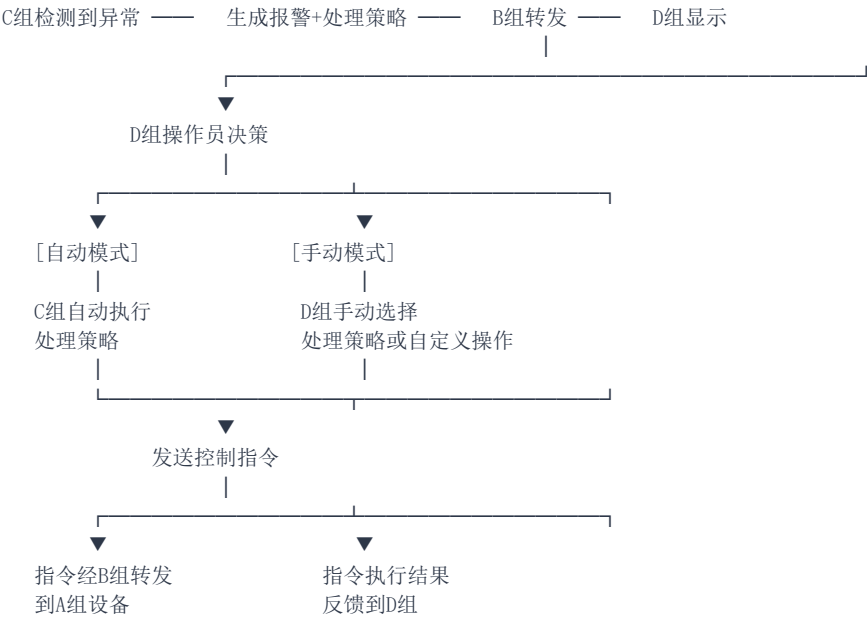
 async Task FetchData()
 {
 using (var client = new HttpClient())
 {
 // 获取AGV位置
 var agvResponse = await client.GetStringAsync($"{apiUrl}/agv/status");
 var agvData = JsonUtility.FromJson<AGVData>(agvResponse);
 UpdateAGVPosition(agvData);

 // 获取传感器数据
 var sensorResponse = await client.GetStringAsync($"{apiUrl}/sensor/latest");
 var sensorData = JsonUtility.FromJson<SensorData>(sensorResponse);
 UpdateSensorDisplay(sensorData);
 }
 }

 void UpdateAGVPosition(AGVData data)
 {
 // 将AGV移动到对应位置
 Vector3 targetPos = GetPositionFromZone(data.position);
 agvObject.transform.position = Vector3.Lerp(
 agvObject.transform.position,
 targetPos,
 Time.deltaTime * 2
);
 }
}
```

任务4：报警处理与策略执行（核心功能）

报警处理流程：



报警弹窗UI（含处理策略）：

|         |                      |          |
|---------|----------------------|----------|
| 警告      | [L3]                 | 机械臂抓取异常  |
| 设备:     | ROBOT-01             |          |
| 时间:     | 10:30:15             |          |
| 描述:     | 连续3次抓取失败             |          |
| 建议处理策略: |                      |          |
| ○ 自动:   | 暂停机械臂, 等待人工检查        |          |
| ● 手动选择: |                      |          |
|         | [重新抓取] [跳过货物] [人工接管] |          |
|         | [复位机械臂] [紧急停止]       |          |
| [确认]    | [执行策略]               | [切换自动模式] |

运行模式切换逻辑：

```
// ModeManager.cs
public class ModeManager : MonoBehaviour
{
 public enum OperationMode { Auto, Manual }
 public OperationMode currentMode = OperationMode.Auto;

 // 切换模式
 public void SwitchMode(OperationMode mode)
 {
 currentMode = mode;

 // 通知B组模式变更
 var msg = new {
 msg_type = "mode_change",
 mode = mode.ToString(),
 timestamp = DateTime.Now
 };
 SendToBackend(msg);
 // 更新UI
 }
}
```

```
 UpdateModeUI(mode);
 }

 // 处理报警
 public void HandleAlert(AlertData alert)
 {
 if (currentMode == OperationMode.Auto)
 {
 // 自动模式：显示报警，但不干预，等C组自动处理
 ShowAlert(alert, readonly: true);
 }
 else
 {
 // 手动模式：显示报警和处理选项
 ShowAlertWithOptions(alert);
 }
 }
}
```

### 手动控制指令发送

```
// ManualControl.cs
public class ManualControl : MonoBehaviour
{
 // 发送控制指令到B组
 public void SendControlCommand(string deviceId, string deviceType, string command, object parameters)
 {
 var cmd = new {
 msg_type = "control_command",
 command_id = Guid.NewGuid().ToString(),
 source = "D组手动控制",
 target_device = deviceId,
 device_type = deviceType,
 command = command,
 params = parameters,
 timestamp = DateTimeOffset.Now.ToString("yyyy-MM-ddTHH:mm:sszzz"),
 require_confirm = true
 };

 // 发送到B组
 httpClient.PostAsync("/api/control/forward", new StringContent(JsonConvert.SerializeObject(cmd)));

 // 记录操作日志
 LogManualOperation(cmd);
 }

 // 机械臂控制示例
 public void RobotReset() => SendControlCommand("ROBOT-01", "robot", "robot_reset", null);
 public void RobotPause() => SendControlCommand("ROBOT-01", "robot", "robot_pause", null);
 public void RobotResume() => SendControlCommand("ROBOT-01", "robot", "robot_resume", null);
 public void RobotEmergencyStop() => SendControlCommand("ROBOT-01", "robot", "robot_stop", null);

 // AGV控制示例
 public void AGVMoveTo(string position) => SendControlCommand("AGV-01", "agv", "agv_move_to", new { target = position });
 public void AGVCharge() => SendControlCommand("AGV-01", "agv", "agv_charge", null);

 // 环境设备控制
 public void FanOn() => SendControlCommand("FAN-01", "env", "fan_on", null);
 public void ExhaustOn() => SendControlCommand("EXHAUST-01", "env", "exhaust_on", null);
 public void BuzzerOn() => SendControlCommand("BUZZER-01", "env", "buzzer_on", null);
}
```

## 任务5：自动巡检动画与摄像头集成

**<b>自动巡检动画</b>:**

```
// PatrolCamera.cs
public class PatrolCamera : MonoBehaviour
{
 public Transform[] patrolPoints; // 巡检路径点
 public float patrolSpeed = 2.0f;
 public float waitTime = 2.0f;

 private int currentPoint = 0;
 private bool isPatrolling = true;

 void Update()
 {
 if (!isPatrolling) return;

 // 移动到下一个巡检点
 Transform target = patrolPoints[currentPoint];
 transform.position = Vector3.MoveTowards(
 transform.position,
 target.position,
 patrolSpeed * Time.deltaTime
);

 // 到达后转向下一个点
 if (Vector3.Distance(transform.position, target.position) < 0.1f)
 {
 StartCoroutine(WaitAndMoveNext());
 }
 }

 IEnumerator WaitAndMoveNext()
 {
 isPatrolling = false;

 // 在该点执行巡检动作（如环顾四周）
 yield return StartCoroutine(LookAround());

 // 等待一段时间
 yield return new WaitForSeconds(waitTime);

 // 移动到下一个点
 currentPoint = (currentPoint + 1) % patrolPoints.Length;
 isPatrolling = true;
 }

 IEnumerator LookAround()
 {
 // 左右环顾动画
 Quaternion originalRot = transform.rotation;
 for (int i = 0; i < 3; i++)
 {
 transform.Rotate(0, 30, 0);
 yield return new WaitForSeconds(0.5f);
 transform.Rotate(0, -60, 0);
 yield return new WaitForSeconds(0.5f);
 }
 transform.rotation = originalRot;
 }
}
```

**<b>摄像头实时画面集成</b>:**

```
// CameraFeed.cs
```

```
public class CameraFeed : MonoBehaviour
{
 public RawImage cameraDisplay; // UI上的RawImage组件
 public string cameraUrl = "http://192.168.1.100:8080/video_feed";

 private VideoPlayer videoPlayer;

 void Start()
 {
 // 方式1: 使用Unity VideoPlayer播放RTSP流
 videoPlayer = gameObject.AddComponent<VideoPlayer>();
 videoPlayer.url = cameraUrl;
 videoPlayer.renderMode = VideoRenderMode.RenderTexture;
 videoPlayer.targetTexture = new RenderTexture(640, 480, 24);
 cameraDisplay.texture = videoPlayer.targetTexture;
 videoPlayer.Play();

 // 方式2: 使用OpenCV for Unity (更灵活)
 // 可以叠加识别框、标签等
 }

 // 切换摄像头视角
 public void SwitchCamera(string cameraId)
 {
 // 机械臂摄像头、全景摄像头、AGV摄像头等
 string[] cameraUrls = {
 "http://192.168.1.101:8080/video_feed", // 机械臂摄像头
 "http://192.168.1.102:8080/video_feed", // 全景摄像头
 "http://192.168.1.103:8080/video_feed" // AGV摄像头
 };

 videoPlayer.url = cameraUrls[int.Parse(cameraId)];
 videoPlayer.Play();
 }
}
```

◁巡检点与摄像头对应关系▷:

| 巡检点     | 位置    | 对应摄像头    | 巡检内容   |
|---------|-------|----------|--------|
| Point 1 | 入库区上方 | 全景摄像头    | 查看入库情况 |
| Point 2 | 输送带上方 | 机械臂摄像头   | 查看货物识别 |
| Point 3 | 分拣区A  | 固定摄像头    | 查看分拣状态 |
| Point 4 | 分拣区B  | 固定摄像头    | 查看分拣状态 |
| Point 5 | AGV路径 | AGV车载摄像头 | 查看搬运过程 |
| Point 6 | 出库区   | 固定摄像头    | 查看出库情况 |

任务6: 数据库与异常监测可视化

◁数据库可视化面板▷:

```
// DatabaseVisualization.cs
public class DatabaseVisualization : MonoBehaviour
{
 public string apiBaseUrl = "http://192.168.1.100:8080/api";

 // 实时数据刷新
 async void RefreshData()
 {
 // 分拣统计
```

```
var sortStats = await FetchData($"{apiBaseUrl}/statistics/sort");
UpdateSortStatsUI(sortStats);

// 异常记录
var alerts = await FetchData($"{apiBaseUrl}/alerts?limit=20");
UpdateAlertsList(alerts);

// 设备状态历史
var deviceHistory = await FetchData($"{apiBaseUrl}/devices/history");
UpdateDeviceChart(deviceHistory);
}

// 异常监测可视化
void UpdateAnomalyVisualization()
{
 // 异常趋势图（24小时）
 DrawAnomalyTrendChart();

 // 设备健康度评分（0-100）
 CalculateDeviceHealthScore();

 // 报警类型分布
 DrawAlertTypePieChart();

 // 处理策略执行记录
 UpdateStrategyExecutionLog();
}
}
```

可视化图表类型：

- 实时折线图：温度、湿度、设备状态随时间变化
- 柱状图：各设备分拣数量对比
- 饼图：报警类型分布、货物类型分布
- 热力图：仓库温湿度分布（结合传感器位置）
- 甘特图：AGV任务执行时间线
- 仪表盘：设备健康度、系统整体状态

## 任务7：与A组协作

需要A组提供：

- 场景尺寸（长×宽×高，单位：米）
- 各区域坐标（入库点、分拣点、出库点）
- AGV路径关键点坐标
- 传感器安装位置
- 摄像头RTSP流地址
- 控制指令协议（如何控制机械臂、AGV等）

坐标系约定：

Unity坐标系：

- X轴：仓库长度方向（0 ~ 2.0m）
- Z轴：仓库宽度方向（0 ~ 1.5m）
- Y轴：高度方向（0 ~ 0.5m）
- 原点：入库区左下角

## 5.3 交付物



| 交付物       | 截止时间  | 验收标准        |
|-----------|-------|-------------|
| Unity场景搭建 | 7月15日 | 包含所有区域和设备模型 |
| 数据面板UI    | 7月20日 | 能显示实时数据     |
| 数据绑定      | 7月25日 | 3D场景与数据同步   |
| 报警可视化     | 7月30日 | 报警弹窗、设备高亮   |
| 技术文档      | 8月5日  | 场景说明、操作手册   |

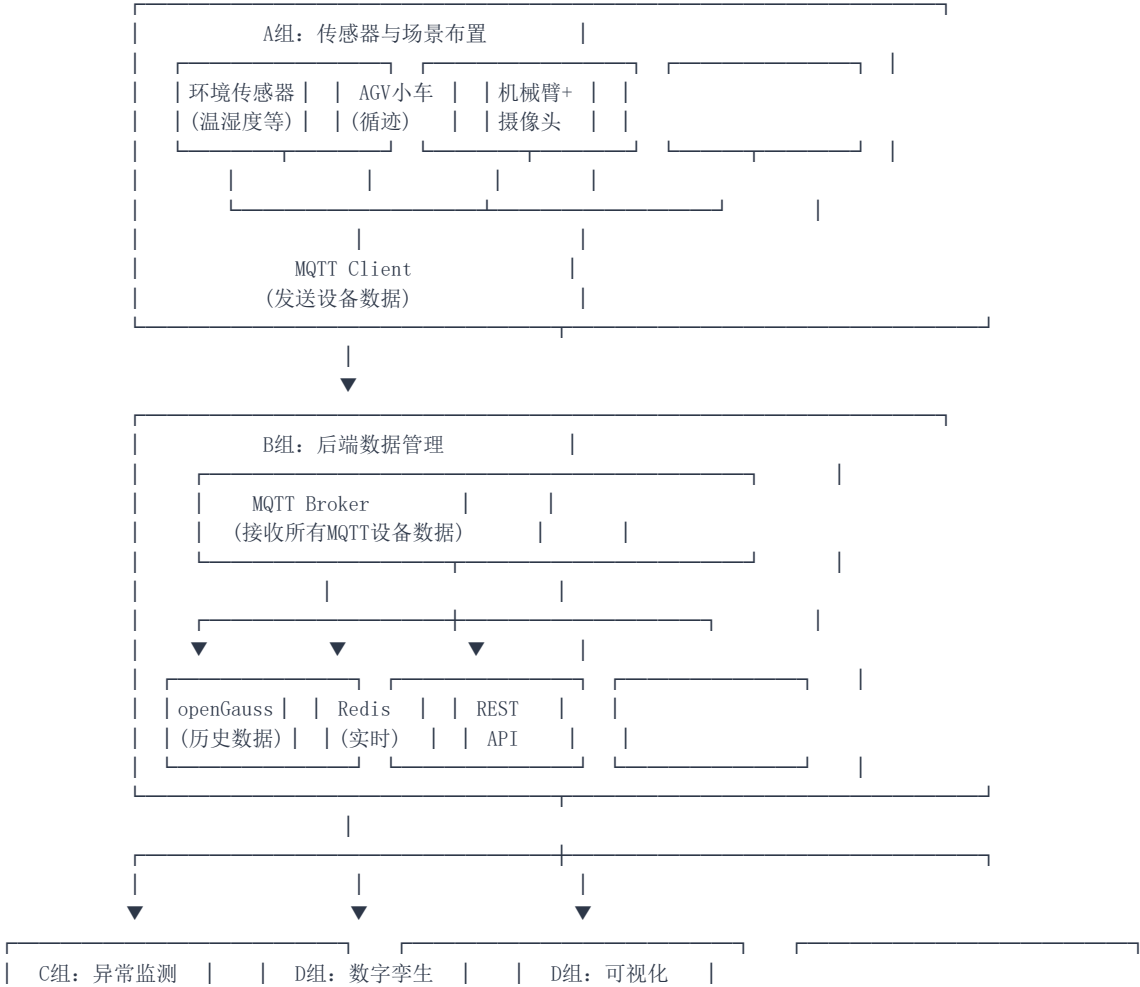
5.4 开发提示

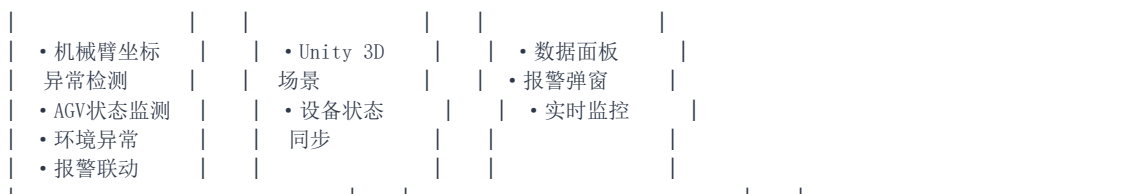
并行开发策略：

- 先用Cube代替真实模型，后期替换
- 数据先用随机数模拟，等B组API就绪后对接
- Unity和Web端可以分开开发，后期嵌入

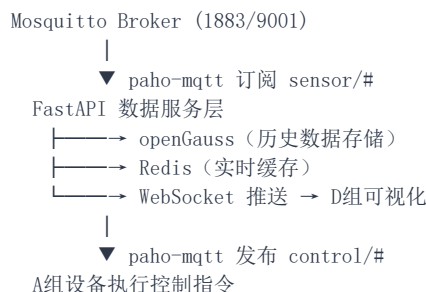
六、组间协作规范

6.1 协作关系图





＜b＞B组后端数据流转架构＜/b＞：



＜b＞数据流向说明＜/b＞：

- ＜b＞A组 → B组＜/b＞：传感器、AGV、机械臂通过MQTT发送设备数据
- ＜b＞B组 → C组＜/b＞：通过Redis发布订阅，实时推送设备数据
- ＜b＞B组 → D组＜/b＞：通过REST API查询，Redis实时推送
- ＜b＞C组 → B组＜/b＞：发送报警+处理策略
- ＜b＞B组 → D组＜/b＞：转发报警+处理策略
- ＜b＞D组 → B组＜/b＞：发送控制指令（手动模式）
- ＜b＞B组 → A组＜/b＞：转发控制指令到设备
- ＜b＞C组 → B组 → A组＜/b＞：自动模式下的处理策略执行

＜b＞控制流说明＜/b＞：

- ＜b＞自动模式＜/b＞：C组检测异常 → 生成策略 → 经B组转发到A组执行
- ＜b＞手动模式＜/b＞：D组操作员决策 → 发送指令 → 经B组转发到A组执行
- ＜b＞模式切换＜/b＞：D组可切换自动/手动模式，通知C组和B组

## 6.2 每日站会（建议）

＜b＞时间＜/b＞：每天晚上9点（线上/线下）

＜b＞时长＜/b＞：15分钟

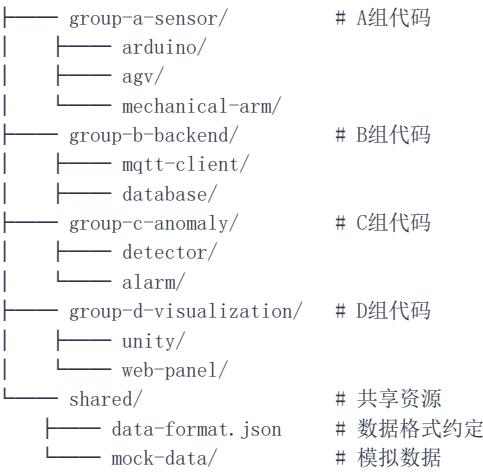
＜b＞内容＜/b＞：

- 今天完成了什么？
- 遇到了什么问题？
- 明天计划做什么？
- 是否需要其他组协助？

## 6.3 代码管理

＜b＞Git仓库结构＜/b＞：





6.4 沟通渠道

| 用途   | 工具           | 说明        |
|------|--------------|-----------|
| 即时通讯 | 微信/钉钉        | 日常沟通      |
| 文档协作 | 腾讯文档/语雀      | 需求文档、接口文档 |
| 代码管理 | Gitee/GitHub | 代码仓库      |
| 任务管理 | 腾讯文档表格       | 任务分配、进度跟踪 |

七、数据格式约定



7.1 统一数据格式

所有组之间传递的数据使用JSON格式，统一编码UTF-8。

7.2 MQTT主题（Topic）与QoS约定

| Topic                                                                       | 方向       | 说明                              | QoS   |
|-----------------------------------------------------------------------------|----------|---------------------------------|-------|
| <font face="SimSun" color="#c7254e"><b>sensor/{type}/{deviceId}</b></font>  | A组→B组    | 传感器数据上报（env/agv/robot）          | QoS 1 |
| <font face="SimSun" color="#c7254e"><b>control/{type}/{deviceId}</b></font> | B组→A组    | 控制指令下发（env/agv/robot）           | QoS 1 |
| <font face="SimSun" color="#c7254e"><b>alert/{level}</b></font>             | C组→B组→D组 | 异常报警（level: info/warn/critical） | QoS 1 |
| <font face="SimSun" color="#c7254e"><b>system/status</b></font>             | 全组       | 心跳与系统状态广播，每30s                  | QoS 0 |
| <font face="SimSun" color="#c7254e"><b>system/command</b></font>            | D组→B组→A组 | 运行模式切换与全局指令                     | QoS 1 |

<b>说明</b>:

- <font face="SimSun" color="#c7254e"><b>sensor/#</b></font>: A组所有设备数据统一订阅前缀, B组通过 <font face="SimSun" color="#c7254e"><b>paho-mqtt</b></font> 订阅 <font face="SimSun" color="#c7254e"><b>sensor/#</b></font> 接收全部设备数据
- <font face="SimSun" color="#c7254e"><b>control/#</b></font>: B组向A组设备下发控制指令的统一前缀
- QoS 1 (至少一次) 用于传感器数据和报警, 确保不丢失; QoS 0 (最多一次) 用于心跳等可丢失信息
- Mosquitto Broker默认支持通配符 <font face="SimSun" color="#c7254e"><b>#</b></font> (多层) 和 <font face="SimSun" color="#c7254e"><b>+</b></font> (单层), 建议B组订阅 <font face="SimSun" color="#c7254e"><b>sensor/#</b></font>, C组订阅 <font face="SimSun" color="#c7254e"><b>alert/#</b></font>

### 7.3 传感器数据格式

```
{
 "msg_type": "sensor_data",
 "device_id": "ENV-01",
 "device_type": "env_sensor",
 "timestamp": "2025-06-15T10:30:00+08:00",
 "payload": {
 "temperature": 26.5,
 "humidity": 65.0,
 "gas_ppm": 150
 },
 "seq": 1001
}
```

### 7.4 AGV数据格式

```
{
 "msg_type": "agv_status",
 "device_id": "AGV-01",
 "device_type": "agv",
 "timestamp": "2025-06-15T10:30:00+08:00",
 "payload": {
 "position": "SORT-AREA-1",
 "position_x": 1.2,
 "position_y": 0.8,
 "battery": 85,
 "status": "running",
 "task_id": "TASK-001"
 },
 "seq": 2001
}
```

### 7.5 AGV控制指令格式

```
{
 "msg_type": "control_command",
 "command_id": "CMD-002",
 "source": "B组调度服务",
 "target_device": "AGV-01",
}
```

```
 "device_type": "agv",
 "command": "agv_move_to",
 "params": {
 "target": "OUT-AREA-1"
 },
 "timestamp": "2025-06-15T10:30:00+08:00",
 "require_confirm": true,
 "triggered_by_alert": null
 }
}
```

7.6 机械臂数据格式

<b>机械臂动作上报</b>（A组发送给B组）：

```
{
 "msg_type": "robot_action",
 "device_id": "ROBOT-01",
 "device_type": "robot",
 "timestamp": "2025-06-15T10:30:00+08:00",
 "payload": {
 "action": "grasp_complete",
 "cargo_type": "box",
 "confidence": 0.92,
 "source_position": "CONVEYOR-1",
 "target_position": "SORT-A",
 "grasp_success": true,
 "joint_angles": [0, 45, 90, 0],
 "tcp_position": [120, 80, 50],
 "error_code": 0
 },
 "seq": 3001
}
```

<b>字段说明</b>：

| 字段              | 类型      | 说明                                                        |
|-----------------|---------|-----------------------------------------------------------|
| action          | string  | 动作类型：grasp_start/grasp_complete/place_complete/idle/error |
| cargo_type      | string  | 货物类型：book/bottle/box/unknown                              |
| confidence      | float   | 识别置信度（0-1）                                                |
| source_position | string  | 抓取位置                                                      |
| target_position | string  | 放置位置                                                      |
| grasp_success   | boolean | 抓取是否成功                                                    |
| joint_angles    | array   | 4个关节角度（度）                                                 |
| tcp_position    | array   | 末端执行器坐标[x, y, z]（mm）                                      |
| error_code      | int     | 错误码，0表示正常                                                 |

7.7 报警与处理策略数据格式（C组→B组→D组）

<b>报警数据格式</b>：

```
{
 "msg_type": "alert_with_strategy",
 "alert_id": "ALERT-001",
 "alert_type": "robot_joint_abnormal",
 "alert_level": 3,
}
```

```

 "device_id": "ROBOT-01",
 "message": "joint2角度异常: 200° ",
 "status": "waiting_confirm",
 "timestamp": "2025-06-15T10:30:00+08:00",
 "payload": {
 "joint_index": 2,
 "current_value": 200,
 "expected_range": [0, 180]
 },
 "strategy": {
 "auto_action": "robot_pause",
 "description": "机械臂关节角度异常, 建议复位或检查",
 "manual_options": [
 {
 "label": "复位机械臂",
 "command": "robot_reset",
 "params": null
 },
 {
 "label": "忽略继续",
 "command": "robot_resume",
 "params": null
 },
 {
 "label": "紧急停止",
 "command": "robot_stop",
 "params": null
 }
]
 },
 "mode": "auto"
 }
}
```

<b>字段说明</b>:

| 字段                      | 类型     | 说明                                               |
|-------------------------|--------|--------------------------------------------------|
| status                  | string | 状态: active/waiting_confirm/acknowledged/resolved |
| strategy.auto_action    | string | 自动执行的动作                                          |
| strategy.description    | string | 处理建议描述                                           |
| strategy.manual_options | array  | 手动处理选项列表                                         |
| mode                    | string | 当前模式: auto/manual                                |

7.8 控制指令数据格式 (D组/C组→B组→A组)

<b>统一控制指令格式</b>:

```

{
 "msg_type": "control_command",
 "command_id": "CMD-001",
 "source": "D组手动控制",
 "target_device": "ROBOT-01",
 "device_type": "robot",
 "command": "robot_reset",
 "params": null,
 "timestamp": "2025-06-15T10:30:00+08:00",
 "require_confirm": true,
 "triggered_by_alert": "ALERT-001"
}
```

控制指令类型列表：

| 设备类型 | command       | 说明    | 参数示例                 |
|------|---------------|-------|----------------------|
| 机械臂  | robot_reset   | 复位机械臂 | null                 |
| 机械臂  | robot_pause   | 暂停机械臂 | null                 |
| 机械臂  | robot_resume  | 继续运行  | null                 |
| 机械臂  | robot_stop    | 紧急停止  | null                 |
| 机械臂  | robot_retry   | 重新抓取  | null                 |
| AGV  | agv_move_to   | 移动到位置 | {"target": "SORT-A"} |
| AGV  | agv_charge    | 前往充电  | null                 |
| AGV  | agv_pause     | 暂停    | null                 |
| 环境设备 | fan_on        | 开启风扇  | null                 |
| 环境设备 | fan_off       | 关闭风扇  | null                 |
| 环境设备 | exhaust_on    | 开启排风  | {"speed": "high"}    |
| 环境设备 | buzzer_on     | 开启蜂鸣器 | {"duration": 5}      |
| 系统   | system_pause  | 暂停系统  | null                 |
| 系统   | system_resume | 恢复系统  | null                 |

7.9 运行模式切换格式

```
{
 "msg_type": "mode_change",
 "mode": "manual",
 "source": "D组操作员",
 "timestamp": "2025-06-15T10:30:00+08:00",
 "reason": "处理机械臂异常"
}
```

7.10 控制指令执行结果反馈（A组→B组→D组）

```
{
 "msg_type": "command_result",
 "command_id": "CMD-001",
 "target_device": "ROBOT-01",
 "device_type": "robot",
 "command": "robot_reset",
 "status": "success",
 "message": "机械臂复位成功",
 "timestamp": "2025-06-15T10:30:05+08:00"
}
```

7.11 模拟数据生成器（B组提供）

用途：前期开发阶段，B组提供模拟数据供C/D组测试

```
mock_data_generator.py
import json
import random
import time
from datetime import datetime, timezone, timedelta, timezone, timedelta

def generate_robot_action():
```

```

"""生成机械臂动作模拟数据"""
actions = ['grasp_start', 'grasp_complete', 'place_complete', 'idle']
cargo_types = ['book', 'bottle', 'box', 'unknown']
sources = ['CONVEYOR-1', 'CONVEYOR-2']
targets = ['SORT-A', 'SORT-B', 'UNKNOWN-AREA']

return {
 "msg_type": "robot_action",
 "device_id": "ROBOT-01",
 "device_type": "robot",
 "timestamp": datetime.now(timezone(timedelta(hours=8))).isoformat(timespec="seconds"),
 "payload": {
 "action": random.choice(actions),
 "cargo_type": random.choice(cargo_types),
 "confidence": round(random.uniform(0.7, 0.99), 2),
 "source_position": random.choice(sources),
 "target_position": random.choice(targets),
 "grasp_success": random.choice([True, True, True, False]), # 75%成功率
 "joint_angles": [
 random.randint(-90, 90),
 random.randint(0, 180),
 random.randint(0, 180),
 random.randint(-90, 90)
],
 "tcp_position": [random.randint(100, 150), random.randint(50, 100), random.randint(30, 60)],
 "error_code": 0
 },
 "seq": random.randint(3000, 4000)
}

def generate_sensor_data():
 """生成传感器模拟数据"""
 return {
 "msg_type": "sensor_data",
 "device_id": f"ENV-{random.randint(1, 4):02d}",
 "device_type": "env_sensor",
 "timestamp": datetime.now(timezone(timedelta(hours=8))).isoformat(timespec="seconds"),
 "payload": {
 "temperature": round(random.uniform(20, 35), 1),
 "humidity": round(random.uniform(40, 80), 1),
 "gas_ppm": random.randint(100, 400)
 },
 "seq": random.randint(1000, 2000)
 }

def generate_agv_data():
 """生成AGV模拟数据"""
 positions = ["INBOUND", "SORT-AREA-1", "SORT-AREA-2", "OUTBOUND", "CHARGE"]
 return {
 "msg_type": "agv_status",
 "device_id": "AGV-01",
 "device_type": "agv",
 "timestamp": datetime.now(timezone(timedelta(hours=8))).isoformat(timespec="seconds"),
 "payload": {
 "position": random.choice(positions),
 "position_x": round(random.uniform(0.5, 2.0), 2),
 "position_y": round(random.uniform(0.3, 1.2), 2),
 "battery": random.randint(10, 100),
 "status": random.choice(["running", "idle", "charging"]),
 "task_id": f"TASK-{random.randint(1, 100):03d}"
 },
 "seq": random.randint(2000, 3000)
 }

```



```
使用示例
if __name__ == '__main__':
 # 每2秒生成一次机械臂数据
 while True:
 robot_data = generate_robot_action()
 print(json.dumps(robot_data, indent=2, ensure_ascii=False))
 time.sleep(2)
```

## 八、并行开发计划



### 8.1 开发阶段划分

阶段1：环境搭建与需求确认（6月15日 - 6月30日）

- ├── 所有组
  - ├── 确定最终需求
  - ├── 搭建开发环境
  - ├── 购买/准备硬件
  - └── 学习相关技术
- └── 里程碑：环境就绪，需求冻结

阶段2：基础功能开发（7月1日 - 7月15日）

- ├── A组：传感器采集、AGV基础控制
- ├── B组：MQTT Broker & Client、数据库
- ├── C组：异常检测规则、报警逻辑
- └── D组：Unity场景搭建、基础UI
- └── 里程碑：各组基础功能可用

阶段3：集成联调（7月16日 - 7月31日）

- ├── A+B：传感器数据接入后端
- ├── B+C：异常检测接入报警
- ├── B+D：数据接入可视化
- ├── A+D：场景坐标对齐
- └── 里程碑：系统基本跑通

阶段4：优化完善（8月1日 - 8月15日）

- ├── 性能优化
- ├── Bug修复
- ├── 文档完善
- └── 里程碑：系统稳定运行

阶段5：演示准备（8月16日 - 比赛日）

- ├── 演示脚本编写
- ├── 演示视频录制
- ├── 答辩PPT制作
- └── 里程碑：比赛就绪

### 8.2 各组并行开发时间线

| 时间          | A组           | B组                      | C组          | D组         |
|-------------|--------------|-------------------------|-------------|------------|
| 6. 15-6. 30 | 买硬件、学Arduino | 学Socket基础 + MQTT协议、装虚拟机 | 学Python、定规则 | 学Unity、找资源 |
| 7. 1-7. 5   | 传感器程序        | MQTT Broker             | 检测规则编码      | 场景搭建       |
| 7. 6-7. 10  | AGV控制        | openGauss部署             | 联动控制        | UI面板       |
| 7. 11-7. 15 | 机械臂调试        | FastAPI接口开发 + WebSocket | 报警系统        | 数据绑定       |

| 时间        | A组   | B组    | C组   | D组    |
|-----------|------|-------|------|-------|
| 7.16-7.20 | 场景搭建 | API开发 | 对接测试 | 动画调试  |
| 7.21-7.25 | 硬件联调 | 性能优化  | 规则调优 | 报警可视化 |
| 7.26-7.31 | 整体联调 | 整体联调  | 整体联调 | 整体联调  |
| 8.1-8.15  | 优化   | 优化    | 优化   | 优化    |
| 8.16-比赛   | 演示准备 | 演示准备  | 演示准备 | 演示准备  |

## 九、开发里程碑



### 9.1 里程碑检查点

| 里程碑     | 日期    | 检查内容      | 通过标准    |
|---------|-------|-----------|---------|
| M1：环境就绪 | 6月30日 | 开发环境、硬件到位 | 所有组环境可用 |
| M2：基础功能 | 7月15日 | 各组基础功能    | 独立功能可用  |
| M3：集成联调 | 7月31日 | 系统集成      | 数据流贯通   |
| M4：系统稳定 | 8月15日 | 优化完善      | 运行稳定    |
| M5：比赛就绪 | 比赛前   | 演示准备      | 可完整演示   |

### 9.2 每周检查清单

每周日晚上9点检查：

- ☐ 本周计划任务完成率
- ☐ 遇到的问题及解决方案
- ☐ 下周计划调整
- ☐ 需要协调的事项

## 十、风险与应对



### 10.1 风险列表

| 风险         | 可能性 | 影响 | 应对措施            |
|------------|-----|----|-----------------|
| 硬件延迟到货     | 中   | 高  | 先用仿真/模拟数据开发     |
| YOLO识别效果差  | 中   | 高  | 简化识别类别，或改用颜色识别  |
| Unity学习成本高 | 中   | 中  | 简化3D效果，用2D面板替代  |
| 麒麟OS环境问题   | 低   | 中  | 先用Ubuntu替代，后期迁移 |
| 组间接口不匹配    | 中   | 高  | 提前约定数据格式，每日同步   |

| 风险     | 可能性 | 影响 | 应对措施      |
|--------|-----|----|-----------|
| 暑假时间冲突 | 高   | 中  | 提前规划，线上协作 |

## 10.2 应急预案

Plan B方案：

- 如果实物设备不到位：使用Gazebo仿真环境
- 如果YOLO效果不好：改用OpenCV颜色/形状识别
- 如果Unity来不及：改用Web端2D可视化（Three.js）
- 如果麒麟OS有问题：先用Ubuntu开发，最后迁移

## 10.3 沟通机制

问题升级路径：

- 组内讨论解决
- 组长间协调
- 全员讨论
- 求助指导老师

## 附录

### A. 推荐学习资源

A组：

- Arduino入门：<https://www.arduino.cc/en/Guide>
- ESP32开发：<https://docs.espressif.com/>
- 小车循迹教程：B站搜索“Arduino循迹小车”

B组：

- MQTT协议（paho-mqtt）：<https://pypi.org/project/paho-mqtt/>
- Python Socket编程：<https://docs.python.org/3/library/socket.html>
- openGauss官方文档：<https://opengauss.org/>
- YOLOv5：<https://github.com/ultralytics/yolov5>

C组：

- Python基础：廖雪峰Python教程
- Redis使用：<https://redis.io/documentation>
- 简单机器学习：吴恩达机器学习（前3周）

D组：

- Unity官方教程：<https://unity.com/learn>
- Unity中文课堂：<https://learn.u3d.cn/>
- C#基础：微软C#文档

B. 推荐硬件购买渠道

- 淘宝：立创商城、优信电子、德飞莱
- 京东：电子元器件自营
- 闲鱼：二手AGV、机械臂（注意验货）

C. 预算参考

| 项目        | 预算           | 说明                         |
|-----------|--------------|----------------------------|
| 传感器套件     | ¥200         | DHT22×4 + MQ-2×2 + ESP32×2 |
| AGV小车     | ¥500         | 循迹小车套件                     |
| 机械臂       | ¥600         | 4DOF机械臂                    |
| 其他配件      | ¥200         | 摄像头、线材、面包板等                |
| <b>总计</b> | <b>¥1500</b> | 可根据实际情况调整                  |

<b>文档版本</b>: 1.0

<b>编写日期</b>: 2025年6月15日

<b>下次评审</b>: 2025年6月30日